

副标题：从 CodeGraph 到 AstraMap，我为什么重新开发一套面向 AI Agent 的代码地图



图 1: 面对百万行级代码库, AstraMap 为 AI Agent 提供精准的工程地图与坐标系

我原本并没有打算重新造一张代码地图。

在治理一个长期演进的百万行级 C/C++ 代码库时，我首先选择了现成方案：用 CodeGraph 把源代码转换成由文件、函数和调用关系组成的图，让原本只能依赖经验和 `grep` 的代码理解，第一次有了可观察的全局结构。

这条路并没有错。

CodeGraph 让我确认了一件事：面对大型代码库，代码地图不是锦上添花，而是基础设施。

但随着使用逐渐深入，我也开始意识到另一个问题：

当代码规模达到百万行，地图“画出来了”，不代表它真的理解了代码。

特别是在 C/C++ 工程里，跨文件声明与定义、头文件包含、宏、条件编译、函数指针、回调注册、平台差异和构建参数，共同决定了代码的真实含义。仅仅从源码表面提取函数名和语法结构，可以生成一张很大的图，却未必能生成一张足够可信的图。

而对于 AI Agent，这个区别至关重要。

因为人类工程师看到一条可疑连线，还会结合经验继续判断；AI Agent 却可能把这条连线直接当成上下文、影响范围甚至修改依据。

这就是我后来决定开发 AstraMap 的起点。

不是因为市场上没有代码地图，而是因为在百万行代码治理和 AI 编程场景里，我需要的不是一张“能展示代码”的地图，而是一张尽可能接近工程真实语义、能够被 AI 直接查询和使用的地图。

一、CodeGraph 让我第一次看见了百万行代码，但也让我看见了代码地图的边界

在没有代码地图之前，理解一个大型遗留项目通常依赖几种方式：

- 搜索函数名；
- 逐层打开文件；
- 查看头文件声明；
- 询问熟悉模块的老工程师；
- 根据目录和命名猜测模块边界；
- 依靠长期积累形成局部心智模型。

这套方法在人还熟悉系统时可以工作，但它有一个明显问题：

代码知识大量存在于人的脑子里，而不是存在于系统里。

当工程达到百万行级别，任何一个人都不可能持续掌握全部关系。人员变化、模块演进、历史兼容和大量条件分支，会不断侵蚀原有认知。

CodeGraph 的价值，是把代码从“文件集合”转换成“关系网络”。

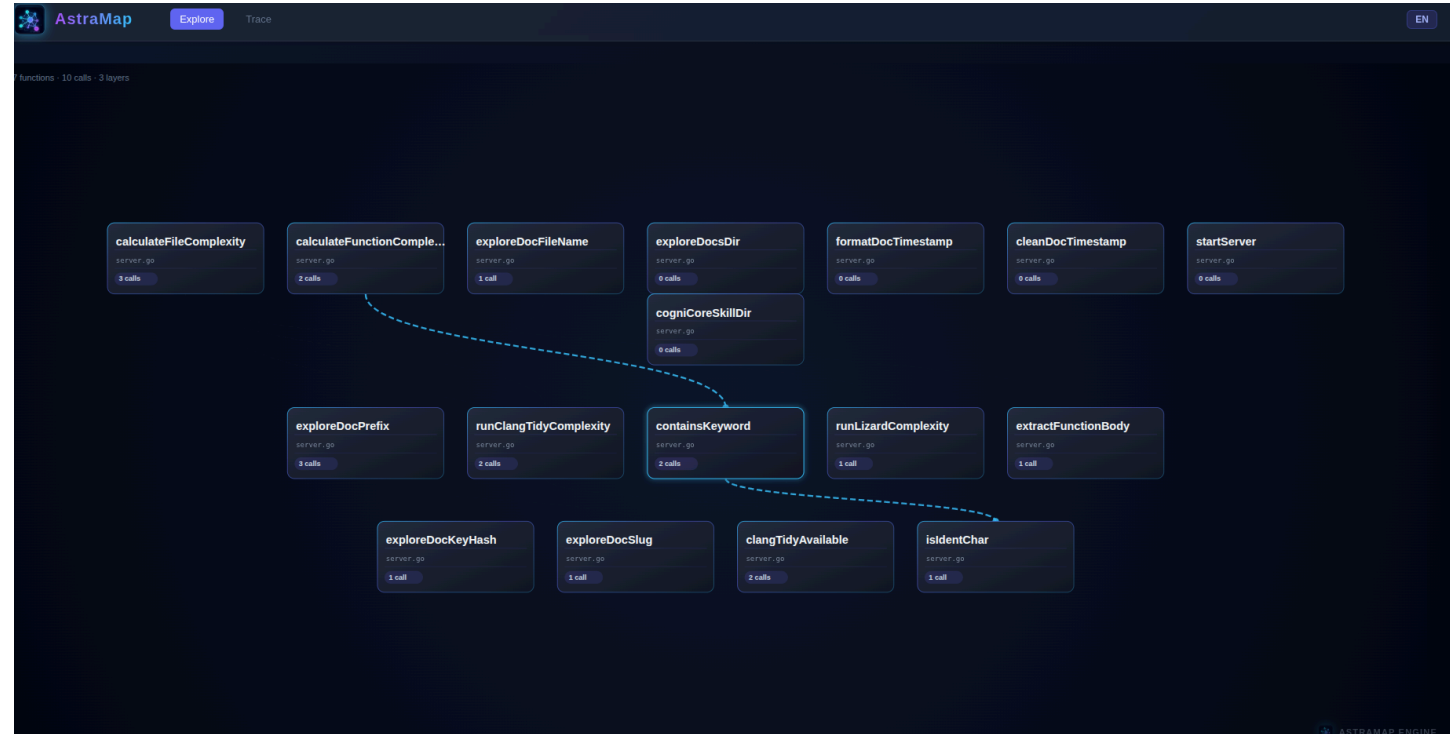


图 2：AstraMap 交互式代码探索视界 —— 将百万行项目转化为可观测的模块与符号拓扑图

过去看到的是：

```
src/
├─ driver/
├─ protocol/
├─ platform/
└─ service/
```

text

有了代码图之后，开始能够看到：

```
文件
↓
函数
↓
调用
↓
模块依赖
```

text

这种变化非常重要。

它让代码治理第一次可以从“我感觉这个模块很复杂”，走向“这个模块的调用扇出、依赖密度和热点函数确实异常”；也让 AI 不再只能通过文件名和关键词盲目寻找上下文。

但真正把它用于百万行 C/C++ 项目后，我发现：语法图能够帮助我们看见代码，却无法单独承担工程级语义判断。

问题并不在于 Tree-sitter 不好。

Tree-sitter 是非常优秀的语法解析基础设施：速度快、部署轻、无需完整编译环境，对多语言的函数、类、结构体和局部调用提取非常有效。

问题在于，百万行工业代码的难点，早已不只是语法。

二、第一道边界：跨文件“同名”，不等于同一个符号

小型项目里，搜索一个函数名，通常能很快找到目标。

但在大型 C/C++ 工程中，同名符号非常常见：

```
init();
open();
send();
read();
update();
```

C

这些名称可能分布在不同模块、不同平台实现、不同静态作用域里。一个 `send()` 可能是网络驱动发送，也可能是消息队列发送，还可能只是某个测试桩的局部函数。

纯语法分析能够识别：

这里出现了一次名为 `send` 的调用。

但它未必能够可靠回答：

这一次调用，究竟指向哪个定义？

C/C++ 还经常把声明和实现拆开：

packet.h	声明	process_packet	
packet.c	实现	process_packet	
rx.c	调用	process_packet	
platform.c	在特定宏条件下替换实现		text

对人来说，这几份文件属于同一个工程语义实体；对只看局部语法的工具来说，它们可能只是几个拼写相同或相近的节点。

一旦跨文件符号绑定不够准确，后面的能力都会被放大污染：

- callers 不可信；
- callees 不完整；

- 调用链可能断裂或错连；
- 影响分析出现大量假阳性；
- 死代码分析把可达代码判成不可达；
- AI 获得了一批“看起来相关、实际上无关”的上下文。

这也是我在使用 CodeGraph 时逐渐形成的第一个判断：

代码地图的关键不在于有多少节点和边，而在于每一条边是否值得信任。

对于人类浏览，模糊关系有时仍然有参考价值；对于自动修改代码的 AI Agent，一条错误关系可能会把整个任务带向错误方向。

三、第二道边界：C/C++ 的真实代码，不只存在于源文件表面

百万行 C/C++ 项目最难处理的部分，往往不是普通函数，而是编译上下文。

1. 宏让“看到的代码”和“参与编译的代码”不是一回事

例如：

```
#define REGISTER_HANDLER(name) \  
    register_handler(#name, handle_##name)  
  
REGISTER_HANDLER(packet);
```

C

源码表面出现的是 `REGISTER_HANDLER(packet)`，真正参与语义分析的却包含 `register_handler`、`handle_packet` 以及宏参数拼接后的结果。

再例如：

```
#define DRIVER_CALL(dev, op, ...) \  
    ((dev)->ops->op(__VA_ARGS__))
```

C

业务代码里可能只有：

```
DRIVER_CALL(dev, send, packet);
```

C

但真正关心的是：

- `dev` 的具体类型是什么；

- `ops` 指向哪张操作表；
- `send` 字段绑定了哪个实现；
- 当前产品配置是否启用了这条路径。

如果代码地图只把宏当成普通文本节点，AI 看到的就不是程序真正的执行关系，而只是源代码的书写形式。

2. 条件编译让同一份仓库拥有多个“代码世界”

大型 C/C++ 项目常见这样的代码：

```
#ifdef CHIP_X
    chip_x_send(packet);
#elif defined(CHIP_Y)
    chip_y_send(packet);
#else
    generic_send(packet);
#endif
```

C

从文件角度看，三个分支都存在。

从某一次具体构建看，真正生效的可能只有一个。

这意味着所谓“谁调用了谁”，不能完全脱离：

- 编译参数；
- 宏定义；
- 目标平台；
- 产品形态；
- 头文件搜索路径。

如果忽略这些条件，图谱会把互斥分支同时连接起来。图会变得更“丰富”，但也更不真实。

所以在 AstraMap 的设计中，C/C++ 的调用边不仅要记录调用关系，还要保留调用点所处的 `#if`、`#ifdef`、`#ifndef` 条件上下文。高精度语义层则依赖有效的 `compile_commands.json`，让索引尽可能接近真实构建环境。

这里有一个非常关键的工程判断：

对于 C/C++，代码地图不是单纯解析源文件，而是在尽可能重建“这份代码如何被编译”。

四、第三道边界：函数指针和回调，把调用关系藏在了初始化路径里

在设备驱动、协议栈、操作系统、中间件和嵌入式软件中，函数指针几乎无处不在。

典型代码可能是：

```
struct driver_ops {  
    int (*open)(struct device *dev);  
    int (*send)(struct device *dev, struct packet *pkt);  
};  
  
static const struct driver_ops ethernet_ops = {  
    .open = ethernet_open,  
    .send = ethernet_send,  
};
```

C

调用处却只有：

```
dev->ops->send(dev, packet);
```

C

从语法上看，被调用者叫 `send`。

但从工程语义上看，它可能指向：

- `ethernet_send`；
- `wifi_send`；
- `virtual_send`；
- 某个平台专用实现；
- 测试环境中的 `mock` 函数。

真实目标不是写在调用表达式里，而是在对象构造、注册表初始化和运行路径中逐渐绑定的。

纯静态分析本来就无法解决所有运行时动态关系，但至少需要比“按名字连接所有同名函数”更克制。

否则代码图很容易退化成一张高噪音网络：



text

看起来信息很多，实际上很难用于工程决策。

因此，AstraMap 一方面利用 SCIP 提供的编译器级符号信息建立确定性骨架，另一方面针对 C 结构体中的函数指针字段构建映射，尽可能把“字段调用”与“候选实现”联系起来；同时，每条边保留来源标记，区分它来自：

- SCIP 确定性语义；
- Tree-sitter 实时语法；
- 启发式推断。

这并不会神奇地解决所有动态绑定，但它至少让 AI 知道：

哪些关系是编译器确认的，哪些关系只是当前最合理的推断。

对于 AI Agent 来说，这种“不确定性可见”比假装所有边同样可靠更重要。

五、第四道边界：百万行项目里，错误地索引内容，比少索引一些内容更危险

代码地图规模变大后，还有一个很容易被忽略的问题：什么不应该进入地图？

大型仓库中不仅有手写业务代码，还可能包含：

- 第三方依赖；
- 自动生成代码；
- 编译输出；
- 缓存目录；
- 临时文件；
- 压缩或合并后的源码；
- 测试生成物；
- 多套重复 SDK；
- 历史废弃副本。

如果这些内容全部进入图谱，会发生什么？

第一，节点和边数量急剧膨胀。

第二，同名符号成倍增加。

第三，AI 搜索时会优先命中生成代码或第三方代码。

第四，模块依赖、复杂度和热点统计都会被污染。

第五，真正承载业务语义的手写代码反而被淹没。

这让我逐渐明确了一条 AstraMap 的索引原则：

代码地图只应该索引手写的、承载业务语义的源代码。由工具、编译器和构建系统自动生成的产物，默认没有进入代码地图的正当理由。

因此，AstraMap 不把过滤责任全部推给用户配置，而是内置生态感知规则：

- Go 排除 `vendor` 和缓存；
- Node.js 排除 `node_modules`、`dist` 等；
- Rust 排除 `target` ；
- CMake 排除 `build`、`cmake-build-*` ；
- Python 排除虚拟环境和缓存；
- 自动识别常见 `generated` 标记；
- 同一套过滤规则同时作用于 Tree-sitter 和 SCIP 输入。

用户只需要表达项目自身的例外，而不应该先学习几十个底层过滤参数。

这看似只是“索引前处理”，实际上决定了代码地图的信噪比上限。

百万行治理的第一步，不是把所有东西都装进地图，而是先决定哪些代码值得被地图记住。

六、现有代码地图的问题，不是它们没有价值，而是目标场景发生了变化

回头看，CodeGraph 并不是一个失败的选择。

它快速、轻量、容易部署，非常适合：

- 中小型项目的结构浏览；
- 代码目录和符号关系探索；
- 不具备完整编译环境时的降级分析；
- 快速为 AI 提供比 `grep` 更好的初始上下文。

问题是，当我们的目标从“看见代码”升级到以下任务时，要求发生了变化：

- 治理百万行遗留代码；
- 精确追踪跨文件调用；
- 评估接口修改的传播范围；
- 自动生成测试范围；

- 支撑智能代码审查；
- 支撑缺陷修复和特性深测；
- 让 AI Agent 直接基于图谱采取行动。

此时，代码地图不能只有覆盖率，还要有可信度；不能只有全量图，还要能按任务裁剪；不能只给人看，还要能被 AI 结构化查询；不能只在索引完成时准确，还要跟得上 AI 修改代码的节奏。

这就是 AstraMap 真正的战场。

它不是要证明 Tree-sitter 没有价值，也不是简单把 CodeGraph 重写一遍。

它要解决的是：

如何在语义精度、实时更新、工程可部署性和 AI Agent 可消费性之间，找到一个新的平衡。

七、为什么不能只依赖更大的上下文窗口？

有人可能会问：

模型上下文窗口越来越大，为什么不直接把更多代码交给 AI？

因为上下文窗口解决的是“能装多少”，并不自动解决“应该装什么”。

把更多代码放进上下文，可能带来三个结果：

第一，相关代码增加了，无关代码也增加了

AI 为了理解一个函数，可能通过 `grep` 找到几十个同名结果，再读取多个大文件。上下文确实更大了，但其中充满同名符号、重复实现、生成代码和互斥平台分支。

第二，文本相邻不等于语义相关

调用者和被调用者可能相隔数十个目录；真正的实现可能通过接口、函数指针或注册表绑定。单纯扩大阅读范围，并不能自动恢复这些关系。

第三，重复搜索会持续燃烧 Token

没有结构化索引时，AI 每次任务都要重新执行：

搜索 → 打开文件 → 判断 → 再搜索 → 再打开文件

text

八、AstraMap 的核心选择：SCIP 决定精度，Tree-sitter 保持实时

在 CodeGraph 的使用过程中，我最不愿意放弃的是 Tree-sitter 的轻量和实时；最希望补上的，是编译器级跨文件语义。

因此，AstraMap 没有走“二选一”，而是采用双层模型。

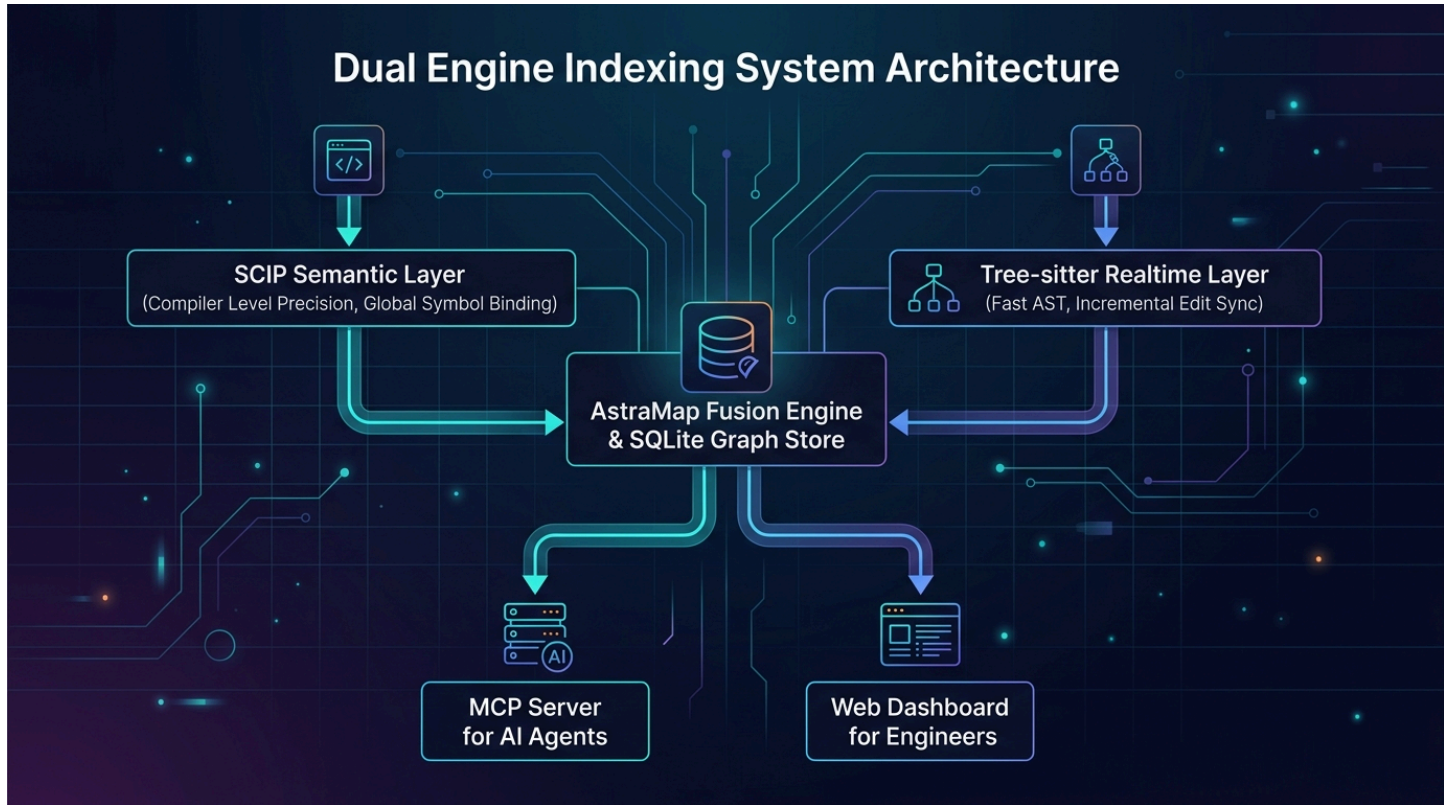


图 3：AstraMap 双层图谱架构——编译器级 SCIP 保证精准绑定，Tree-sitter 保证实时增量响应
两层承担不同职责。

SCIP 语义层回答：“它到底是谁？”

它负责：

- 跨文件符号绑定；
- 全局唯一符号身份；
- 类型和实现关系；
- 重载与命名空间消歧；
- 编译器级引用和调用关系。

对 C/C++ 而言，高精度索引依赖有效的 `compile_commands.json`。AstraMap 不会擅自执行构建命令，而是复用用户已有的构建数据库，对路径进行规范化和过滤，再交给 `scip-clang`。

这是一种克制：

工具应该尊重工程的构建事实，而不是为了“自动化”偷偷改变用户的构建环境。

Tree-sitter 实时层回答：“磁盘上的代码现在变成什么样？”

AI 编程过程中，代码会被连续修改。

如果每保存一次文件都重新生成全量 SCIP 索引，反馈成本难以接受。因此，Tree-sitter 负责：

- 单文件快速解析；
- 哈希级变更检测；
- 新增函数和结构的即时更新；
- SCIP 两次构建之间的降级可用；
- 修复编辑导致的位置漂移。

AstraMap 的合并规则是：

- 同一关系发生冲突时，SCIP 语义优先；
- Tree-sitter 始终反映当前磁盘状态；
- 启发式关系必须明确标注来源；
- 图谱变化后，相关查询缓存同步失效。

这不是简单把两种技术叠在一起，而是明确回答了一个工程问题：

高精度索引通常不够实时，实时解析通常不够精确，AI Agent 同时需要二者。

九、AstraMap 不只是给人看图，而是让 AI 能直接“问路”

传统代码地图的主要交互方式是可视化。

人打开一张图，拖动节点、查看依赖、寻找路径。

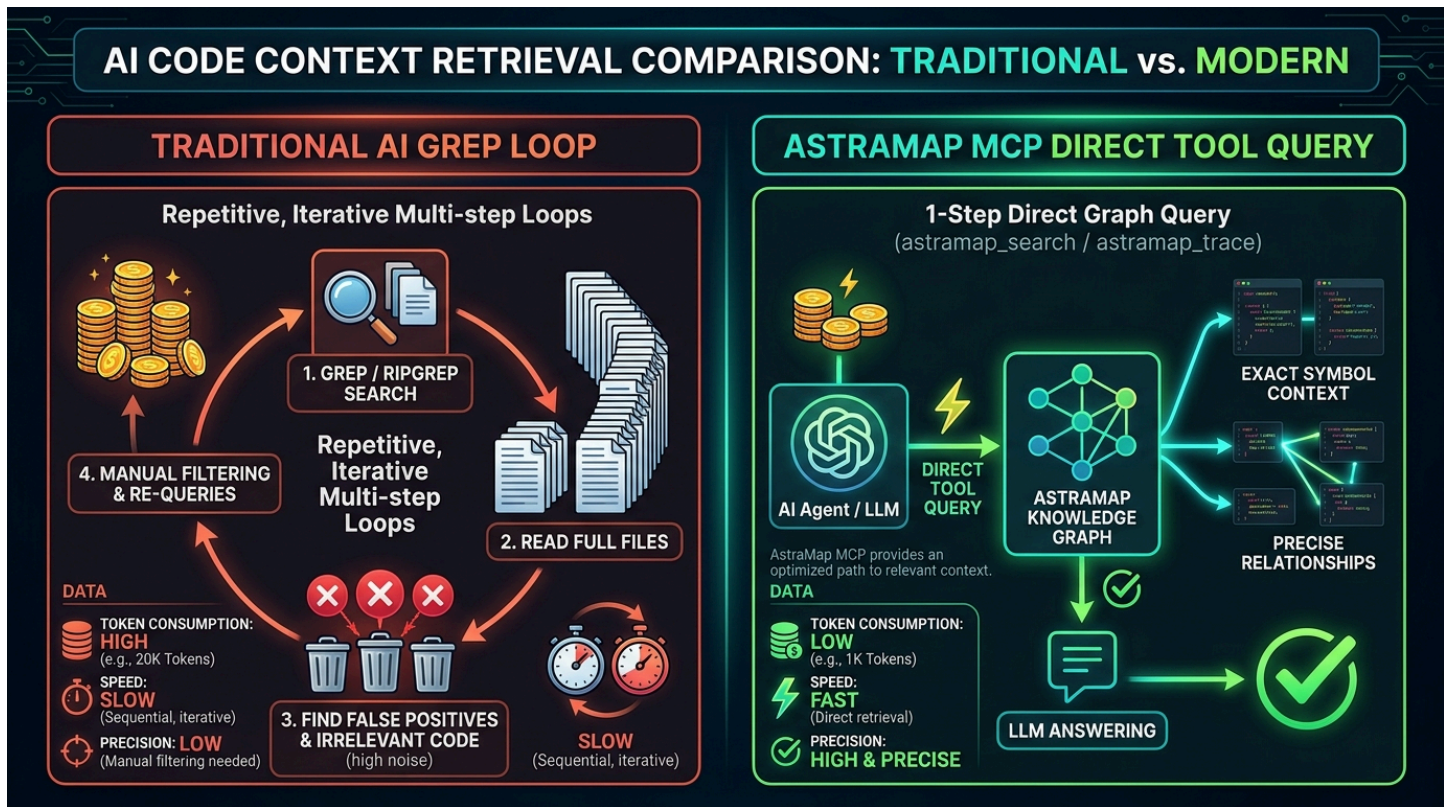


图 4：AI 获取上下文两种范式对比 —— 传统 Grep 循环 (高 Token / 高噪音) vs AstraMap MCP (一步直达 / 高精度)

可视化仍然重要，所以 AstraMap 保留了：

- 项目、目录、文件和函数逐层探索；
- callers 与 callees 邻域；
- 调用路径追踪；
- 模块与文件理解文档；
- 影响图、循环依赖和耦合分析。

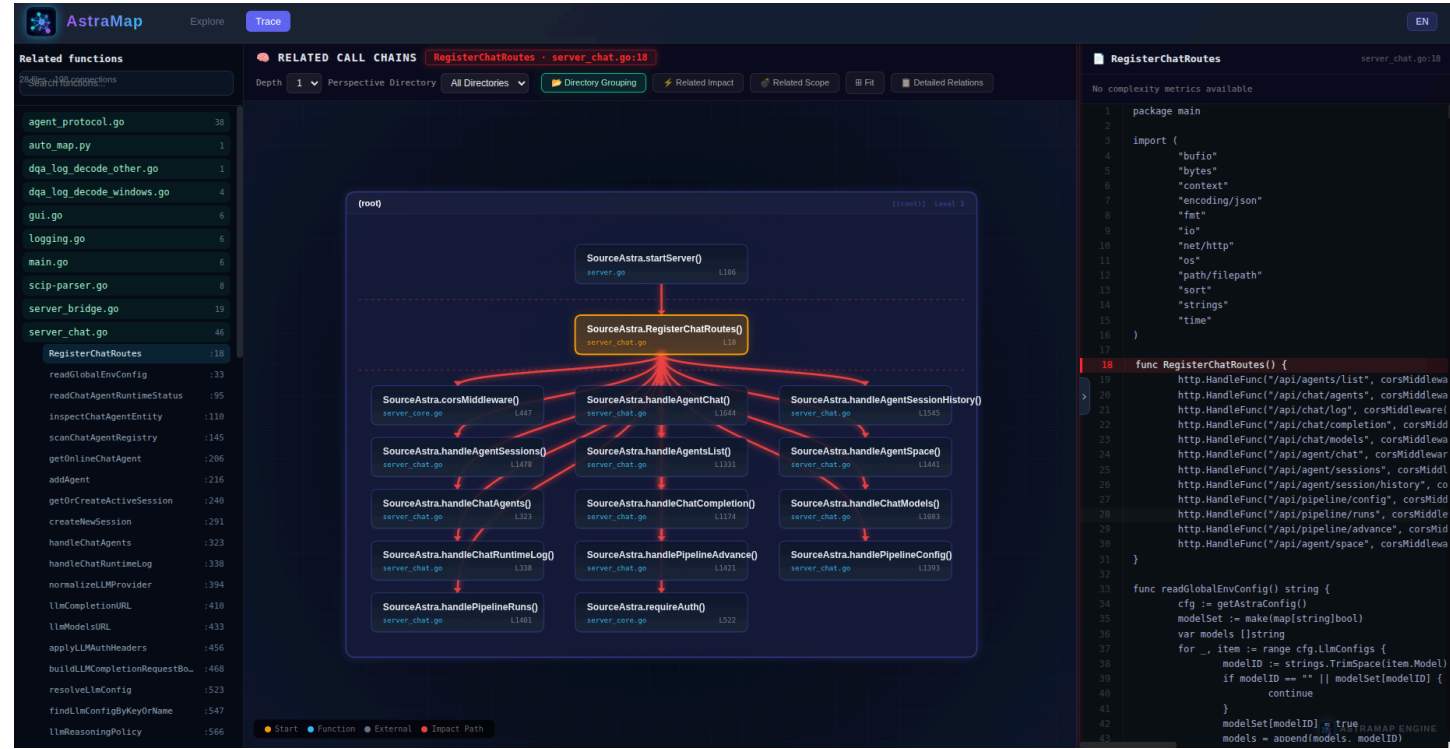


图 5：AstraMap 精确 Call Trace 链路追踪——一键探明跨文件调用传播路径

但 AI Agent 不应该通过“看截图”使用代码地图。

它需要的是结构化查询。

因此 AstraMap 通过 MCP 暴露一组面向任务的工具，例如：

astramap_search	查找符号	text
astramap_node	获取定义、签名和源码片段	
astramap_callers	谁调用了它	
astramap_callees	它调用了谁	
astramap_trace	两个符号之间的调用路径	
astramap_impact	修改后可能影响哪些节点	
astramap_status	当前索引质量和来源覆盖情况	

这会改变 AI 获取上下文的方式。

过去：

grep 函数名 → 打开多个文件 → 排除同名符号 → 继续搜索调用者 → 拼接一批可能相关的上下文

现在：

定位唯一符号 → 查询上下游关系 → 按任务裁剪调用链 → 只读取必要源码

这才是代码地图真正服务 AI Agent 的方式。

地图不是为了画得更漂亮，而是为了让 Agent 少走弯路。

十、从“代码可视化”到“AI 的工程坐标系”

AstraMap 最终想解决的，并不只是“函数跳转更快”。

在我的研发实践中，代码地图会成为多种 AI 工程能力的底座：

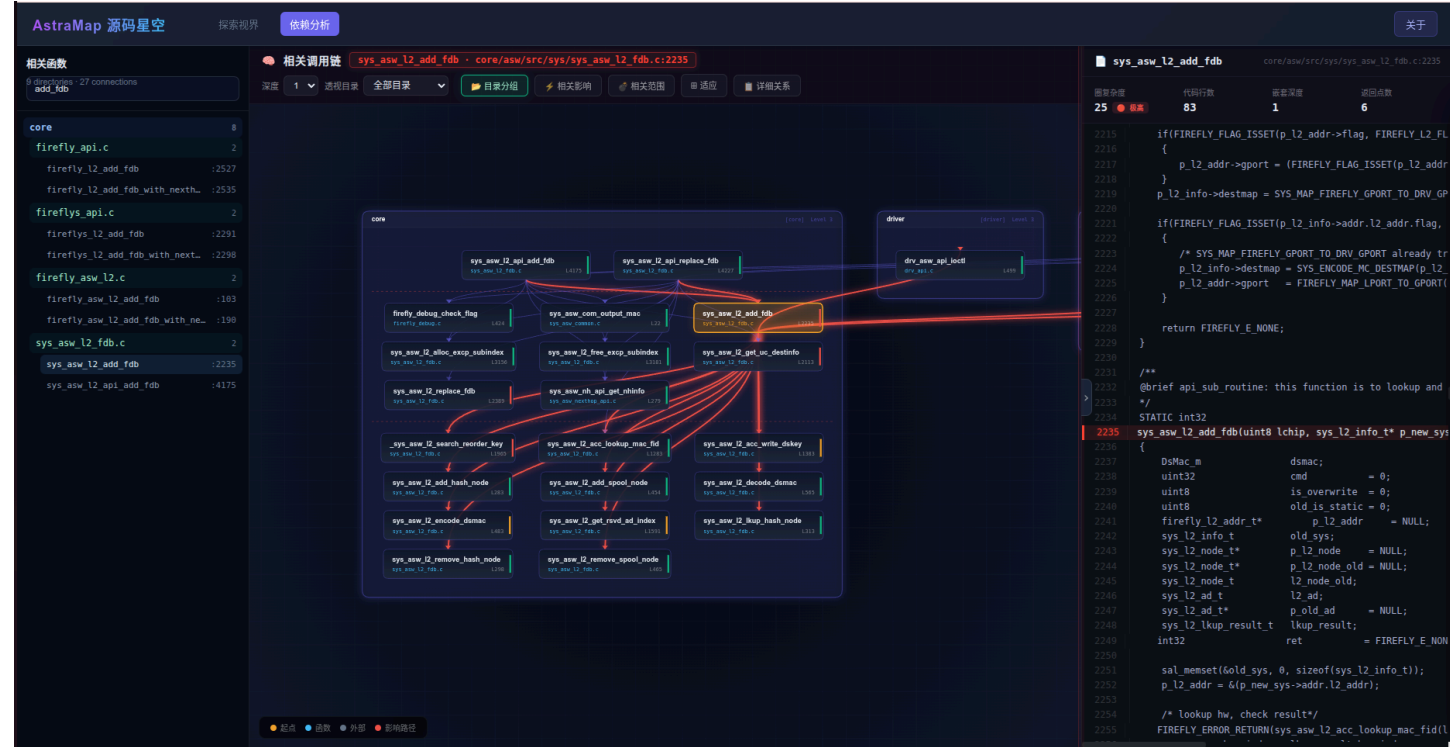


图 6：AstraMap 模块依赖与影响图分析—— 为智能 Code Review 与影响范围评估提供图谱底座

智能代码审查

不是把整个文件交给大模型，而是根据拓扑找到真正相关的调用者、被调用者、接口实现和历史风险区域。

缺陷修复

不仅找到报错位置，还能沿着调用关系寻找根因、评估修改边界，并识别可能需要同步调整的路径。

模块测试与特性深测

从被测接口出发，识别依赖、分支、外部交互和影响范围，帮助 AI 生成更完整的测试计划，而不是只根据单个函数体编写几个表面用例。

代码治理

通过调用图、循环依赖、耦合关系、热点函数和死代码候选，建立百万行代码的治理优先级。

这些任务有一个共同前提：

AI 必须先知道代码位于哪里、与谁相连、关系是否可信，再谈推理和行动。

因此，我越来越确信：

大模型的原生智力只是 AI 编程能力的一部分。

真正决定大型工程交付质量的，是外围工程系统是否为模型提供了：

- 干净的输入；
- 精确的上下文；
- 可查询的结构；
- 可验证的结果；
- 可追溯的过程；
- 可持续修正的反馈闭环。

AstraMap 是其中的“代码空间层”。

它负责把百万行代码，从无法整体阅读的文本海洋，转换成 AI 可以定位、探索、追踪和验证的工程坐标系。

十一、我也不想把 AstraMap 描述成一张“万能地图”

工程工具需要诚实面对边界。

AstraMap 并不能静态还原所有运行时行为。

尤其在 C/C++ 中，以下场景仍然存在天然困难：

- 复杂运行时函数指针赋值；
- 动态装载；
- 反射或代码生成；
- 不完整的构建环境；
- 缺失或失真的 `compile_commands.json` ；
- 大量依赖外部配置决定的代码路径。

当 SCIP Provider 不可用时，AstraMap 会降级到 Tree-sitter 和启发式关系，而不是假装仍然拥有编译器级精度。每个节点和边记录 `provenance` ，正是为了让使用者和 AI 知道数据从哪里来、应该信任到什么程度。

我不认为一张代码地图必须在所有场景里做到“绝对正确”。

但它至少应该做到：

1. 尽可能使用权威语义；
2. 无法确定时明确暴露不确定性；
3. 不用错误的确定性误导 AI；
4. 在准确性和实时性之间给出可解释的退化路径。

这也是我从使用现成代码地图，到决定自己开发 AstraMap 的最重要认知变化。

结语：我不是想再画一张图，而是想给 AI 修一套道路系统

从 CodeGraph 到 AstraMap，并不是一个“现成工具不好，所以重新造轮子”的故事。

CodeGraph 帮助我跨出了关键的第一步：让我确认代码地图对于百万行治理和 AI 编程具有真实价值。

但深入到大型 C/C++ 工程后，我看到了一条更深的分界线：

- 语法结构不等于编译语义；
- 同名符号不等于同一个实体；
- 源文件内容不等于实际参与构建的代码；
- 图上有一条边，不等于这条边值得 AI 信任；
- 能让人浏览的地图，不等于能让 AI 采取行动的地图。

所以，我设计 AstraMap 的初衷，不是展示更多节点，也不是单纯减少几次 `grep`。

而是希望回答一个面向 AI 原生研发的问题：

当 AI Agent 真正进入百万行软件工程，我们应该如何为它重建代码世界？

我的答案是：

先用 SCIP 建立可信的语义骨架，用 Tree-sitter 跟踪实时变化，用来源追踪表达确定性，再通过 MCP 把这张地图直接交给 AI。

面对百万行代码，AI 需要的确实不是简单更大的上下文窗口。

因为上下文窗口只能让它一次看到更多文本。

而一张真正懂代码的地图，才能告诉它：

- 自己现在在哪里；
- 这段代码通向哪里；
- 哪些关系真实存在；
- 修改一步之后，哪里可能发生变化。

大模型负责思考，代码地图负责让它不迷路。

这就是 AstraMap 的战场，也是我开发它的初衷。